Python Engineering - Reference Note

__name__, __main__, and the main guard

Simple explanation of why `if __name__ == "__main__":` commonly appears at the end of Python programs.

1. What is __name__?

Python automatically creates a special variable named `__name__` inside every module. Its value depends on how the file was loaded.

Situation	Value of __name__
You run python main.py	'__main__'
Another file does import vertical_motion	'vertical_motion'

2. What does the main guard ask?

```
if __name__ == '__main__':
    main()
```

It asks: **"Is this file the program being executed directly?"** If yes, it calls `main()`. If the file was only imported, the application flow does not start automatically.

3. Why is it commonly used?

Because the same file can act as an executable program and as a reusable module. The main guard prevents import side effects: functions become available without unexpectedly running the primary application flow.

4. Why also define main()?

It wraps the primary program flow, keeps code organized, helps testing, and makes the application entry point clear.

```
def main():
    config = {...}
    results = simulate_vertical_motion(config)
    print(results)

if __name__ == '__main__':
    main()
```

5. Mental model

```
DIRECT EXECUTION
python main.py -> __name__ = '__main__' -> main() RUNS

IMPORT
import main -> __name__ = 'main' -> main() DOES NOT RUN AUTOMATICALLY
```

6. Important distinction

`__main__` is not a mandatory function name and `main()` is not a Python keyword. `__main__` is a special value of `__name__`; naming the primary function `main()` is a widespread convention.

Practical rule: use the main guard when you want a primary flow to run directly while still allowing the file functions to be imported without starting that flow.